**Варианты заданий для лабораторных работ по дисциплине**
**«Качество и надежность программного обеспечения»**

## Программа 1. Численное интегрирование методом Симпсона (в.1)

```
program simp1;
{ integration by Simpson's method }
const tol         = 1.0E-6;
var   sum,upper,lower   : real;

function fx(x: real): real;
begin
  fx:=exp(-x/2)
end;  { function fx }
function dfx(x: real): real;
begin
  dfx:=-(exp(-x/2))/2
end;  { function fx }

procedure simps(
            lower,upper,tol   : real;
            var sum           : real);
 { numerical integration by Simpson's rule }
{ function is fx, limits are lower and upper }
{ with number of regions equal to pieces }
{ partition is delta_x, answer is sum }

var   i           : integer;
      x,delta_x,even_sum,
      odd_sum,end_sum,
      end_cor,sum1      : real;
      pieces            : integer;

begin
  pieces:=2;
  delta_x:=(upper-lower)/pieces;
  odd_sum:=fx(lower+delta_x);
  even_sum:=0.0;
  end_sum:=fx(lower)+fx(upper);
  end_cor:=dfx(lower)-dfx(upper);
  sum:=(end_sum+4.0*odd_sum)*delta_x/3.0;
  repeat
    pieces:=pieces*2;
    sum1:=sum;
    delta_x:=(upper-lower)/pieces;
    even_sum:=even_sum+odd_sum;
    odd_sum:=0.0;
    for i:=1 to pieces div 2 do
      begin
      x:=lower+delta_x*(2.0*i-1.0);
      odd_sum:=odd_sum+fx(x)
      end;
    sum:=(7.0*end_sum+14.0*even_sum+16.00*odd_sum
                        +end_cor*delta_x)*delta_x/15.0;
  until (sum<>sum1) and (abs(sum-sum1)<=abs(tol*sum))
end;  { simps }
begin       { main program }
  lower:=1.0;
  upper:=9.0;
    simps(lower,upper,tol,sum);
  writeln;
```

```
  writeln(chr(7),'area= ',sum)
end.
```

**Программа 2. Интегрирование методом трапеций (в.1).**

```
program trap;
{ integration by the trapezoidal rule }

const tol         = 1.0E-6;
var   sum,upper,lower   : real;

function fx(x: real): real;
{ find f(x)=1/x }
{ watch out for x=0 ! }
begin
  fx:=1.0/x
end;

procedure trapez(lower,upper,tol: real;
            var sum           : real);
{ numerical integration by the trapezoid method }
{ function is FX, limits are LOWER and UPPER }
{ with number of regions equal to PIECES }
{ fixed partition is DELTA_X, answer is SUM }

var   pieces,i                : integer;
      x,delta_x,end_sum,mid_sum,sum1      : real;
begin
  pieces:=1;
  delta_x:=(upper-lower)/pieces;
  end_sum:=fx(lower)+fx(upper);
  sum:=end_sum*delta_x/2.0;
  writeln('    1',sum);
  mid_sum:=0.0;
  repeat
    pieces:=pieces*2;
    sum1:=sum;
    delta_x:=(upper-lower)/pieces;
    for i:=1 to pieces div 2 do
    begin
      x:=lower+delta_x*(2.0*i-1.0);
      mid_sum:=mid_sum+fx(x)
    end;
  sum:=(end_sum+2.0*mid_sum)*delta_x*0.5;
  writeln(pieces:5,sum)
  until abs(sum-sum1)<=abs(tol*sum)
end;        { TRAPEZ }

begin       { main program }
  ClrScr;
  lower:=1.0;
  upper:=9.0;
  writeln;
  trapez(lower,upper,tol,sum);
  writeln;
  writeln(chr(7),'area=',sum)
end.
```

**Программа 3. Вычисление корня методом Ньютона.**

```
program newdr;
var   x,x2  : real;
      alldone     : boolean;
      error : boolean;
```

```
procedure func(x: real;
            var fx,dfx: real);

{ the vapor pressure of lead }
const
   a = 18.19;
   b = -23180.0;
   c = -.8858;
   logp = -4.60517  { ln(.01) }
begin
  fx:= a + b/x + c*ln(x) - logp;
  dfx:= -b/(x*x) + c/x
end;  { func }

procedure newton(var x: real);
const       tol   = 1.0E-6;
var   fx,dfx,dx,x1:     real;
begin { newton }
  repeat
    x1:=x;
    func(x,fx,dfx);
    if(abs(dfx)<tol) then
       begin
          if(dfx>=0.0) then dfx:=tol
          else dfx := -tol
       end;
    dx:=fx/dfx;
    x:=x1-dx;
    writeln('x=',x1,',fx=',fx,',dfx=',dfx);
  until abs(dx)<=abs(tol*x)
end;  { newton }

begin       { main program }
  alldone:=false;
  repeat
    writeln;
    write('First guess (999. to exit): ');      { first guess }
    readln(x);
    if x=999. then alldone:=true
    else
      begin
      newton(x);
      writeln;
      writeln('The solution is ',x);
      writeln
      end
  until alldone
end.
```

## Программа 4. Сортировка методом Шелла (в.1)

```
program Shell_sort;
{ Shell-Metzner sort }
{ Adapted from 'Programming in pascal',
      P. Grogono, Addison-Wesley, 1980 }

const   max     = 80;

type    ary     = array[1..max]of real;

var     x         : ary;
        i,n       : integer;
```

```
procedure {shell} sort(var a: ary; n: integer);

var   done  : boolean;
      jump,i,j: integer;
procedure swap(var p,q: real);
var   hold  : real;

begin
  hold:=p;
  p:=q;
  q:=hold
end;        { swap }

begin
  jump:=n;
  while jump>1 do
    begin
      jump:=jump div 2;
      repeat
      done:=true;
      for j:=1 to n do
        begin
          i:=j+jump;
          if a[j]>a[i] then
            begin
            swap(a[j],a[i]);
            done:=false
          end           { if }
      end         { for }
      until done
  end             { while }
end;        { SORT }

procedure write_arr;
{ print out the answer }
var
  i: integer;

begin
  writeln;
  for i:=1 to n do
      write(x[i]:7:1,' ');
      writeln;
end;        { write_arr }

begin   { MAIN program }
 n:=max;
 randomize;
 for i:=1 to n do
   x[i]:= random(100);
 sort( x,n );
 write_arr
end.
```

**Программа 5. Добавить головную программу и сравнить эффективность двух процедур сортировки.**

```
program bubble_sort1-2;
{ bubble sorting - vers.1 without procedure, vers.2 with swap-procedure }
const   max     = 1000;
type    ary     = array[1..max] of real;

var     x         : ary;
```

```
        i,n       : integer;

procedure { bubble } sort1(var a: ary; n: integer);
var   i,j   : integer;
      hold  : real;

begin       { procedure sort1 }
  for i:=1 to n-1 do
    for j:=i+1 to n do
      begin
      if a[i]>a[j] then
        begin
          hold:=a[i];
          a[i]:=a[j];
          a[j]:=hold
        end
    end           { for }
end;        { procedure sort1 }


procedure {bubble} sort2(var a: ary; n: integer);

var   no_change   : boolean;
      j           : integer;

procedure swap(p,q: real);
var   hold  : real;
begin
  hold:=p;
  p:=q;
  q:=hold
end;        { swap }

begin       { procedure sort2 }
  repeat
    no_change:=true;
    for j:=1 to n-1 do
      begin
      if a[j]>a[j+1] then
        begin
          swap(a[j],a[j+1]);
          no_change:=false
        end
      end   { for }
  until no_change
end;  { procedure sort2 }
```

## Программа 6. Вычисление функции Бесселя 2-го рода (в.1).

```
program bessy1;
{ evaluation of Bessel function of the second kind }
uses crt;

var   x,ordr      : real;
      done  : boolean;

function bessy(x,n: real): real;
{ cylindical bessel function of the second kind }
const small = 1.0E-8;
      euler = 0.57721566;
      pi    = 3.1415926;
      pi2   = 0.63661977;     { 2/pi }
var   j     : integer;
```

```
      x2,sum,sum2,t,t2,
      ts,term,xx,y0,y1,
      ya,yb,yc,ans,a,b,
      sina,cosa         : real;

begin             { function bessy }
  if x<12 then
    begin
      xx:=0.5*x;
      x2:=xx*xx;
      t:=ln(xx)+euler;
      sum:=0.0;
      term:=t;
      y0:=t;
      j:=0;
      repeat
      j:=j+1;
      if j<>1 then sum:=sum+1/(j-1);
      ts:=t-sum;
      term:=-x2*term/(j*j)*(1-1/(j*ts));
      y0:=y0+term
      until abs(term)<small;
      term:=xx*(t-0.5);
      sum:=0.0;
      y1:=term;
      j:=1;
      repeat
      j:=j+1;
      sum:=sum+1/(j-1);
      ts:=t-sum;
      term:=(-x2*term)/(j*(j-1))*((ts-0.5/j)/(ts+0.5/(j-1)));
      y1:=y1+term
      until abs(term)<small;
      y0:=pi2*y0;
      y1:=pi2*(y1-1/x);
      if n=0.0 then ans:=y0
      else if n=1.0 then ans:=y1
      else
      begin       { find y by recursion }
        ts:=2.0/x;
        ya:=y0;
        yb:=y1;
        for j:=2 to trunc(n+0.01) do
          begin
            yc:=ts*(j-1)*yb-ya;
            ya:=yb;
            yb:=yc
          end;
        ans:=yc
      end;
      bessy:=ans;
    end           { x<12 }
  else            { x>11, asymtotic expansion }
    bessy:=sqrt(2/(pi*x))*sin(x-pi/4-n*pi/2)
end;  { function bessy }

begin
  ClrScr;
  done:=false;
  writeln;
  repeat
    write('Order? ');
    readln(ordr);
    if ordr<0.0 then done:=true
```

```
    else
      begin
      repeat
        write('Arg? ');
        readln(x)
      until x>=0.0;
      writeln('Y Bessel is ',bessy(x,ordr))
    end           { if }
  until done
end.
```

Исх.данные: x = 4 ,ordr = 2

## Программа 7. Интегрирование методом Симпсона (в.2).

```
program simp1;
{ integration by Simpson's method }
{ Turbo Pascal cannot pass function names as arguments}

const tol         = 1.0E-6;
var   sum,upper,lower   : real;

function fx(x: real): real;
{ find f(x)=1/x }
{ watch out for x=0 }

begin
  fx:=1.0/x
end;  { function fx }

procedure simps(
            lower,upper,tol   : real;
            var sum           : real);
{ numerical integration by Simpson's rule }
{ function is fx, limits are lower and upper }
{ with number of regions equal to pieces }
{ partition is delta_x, answer is sum }

var   i           : integer;
      x,delta_x,even_sum,
      odd_sum,end_sum,
      sum1        : real;
      pieces            : integer;
begin
  pieces:=2;
  delta_x:=(upper-lower)/pieces;
  odd_sum:=fx(lower+delta_x);
  even_sum:=0.0;
  end_sum:=fx(lower)+fx(upper);
  sum:=(end_sum+4.0*odd_sum)*delta_x/3.0;
  writeln(pieces:5,sum);
  repeat
    pieces:=pieces*2;
    sum1:=sum;
    delta_x:=(upper-lower)/pieces;
    even_sum:=even_sum+odd_sum;
    odd_sum:=0.0;
    for i:=1 to pieces div 2 do
      begin
      x:=lower+delta_x*(2.0*i-1.0);
      odd_sum:=odd_sum+fx(x)
      end;
    sum:=(end_sum+4.0*odd_sum+2.0*even_sum)*delta_x/3.0;
    writeln(pieces:5,sum)
  until (sum<>sum1) and (abs(sum-sum1)<=abs(tol*sum))
```

```
end;  { simps }

begin       { main program }
  ClrScr;
  lower:=1.0;
  upper:=9.0;
  writeln;
  simps(lower,upper,tol,sum);
  writeln;
  writeln(chr(7),'area= ',sum)
end.
```

## Программа 8. Быстрая сортировка (нерекурсивный вариант) – в.1.

```
const   max     = 80;

type    ary     = array[1..max]of real;

var     x         : ary;
        i,n       : integer;

procedure sort(var x: ary; n: integer);
{ a NONRECURSIVE quicksort routine }
{ Adapted from 'Software-Tools',
      B.Kernighan, Addison Wesley, 1976 }

var   left,right  : array[1..20] of integer;
      i,j,sp,mid  : integer;
      pivot       : real;

procedure swap(var p,q: real);
var   hold  : real;

begin
  hold:=p;
  p:=q;
  q:=hold
end;        { swap }


begin
  left[1]:=1;
  right[1]:=n;
  sp:=1;
  while sp>0 do
    begin
      if left[sp]>=right[sp] then sp:=sp-1
      else
      begin
        i:=left[sp];
        j:=right[sp];
        pivot:=x[j];
        mid:=(i+j)div 2;
        if (j-i)>5 then
          if ((x[mid]<pivot)and(x[mid]>x[i]))
            or
            ((x[mid]>pivot)and(x[mid]<x[i]))
              then swap(x[mid],x[j])
          else
            if((x[i]<x[mid])and(x[i]>pivot))
            or ((x[i]>x[mid])and(x[i]<pivot))
              then swap(x[i],x[j]);
    pivot:=x[j];
    while i<j do
```

```
      begin
      while x[i]<pivot do
        i:=i+1;
      j:=j-1;
      while (i<j)and(pivot<x[j]) do
        j:=j-1;
      if i<j then swap(x[i],x[j])
    end;    { while }
    j:=right[sp]; { pivot to i }
    swap(x[i],x[j]);
    if i-left[sp]>=right[sp]-i then
      begin       { put shorter part first }
      left[sp+1]:=left[sp];
      right[sp+1]:=i-1;
      left[sp]:=i+1
      end
    else
      begin
      left[sp+1]:=i+1;
      right[sp+1]:=right[sp];
      right[sp]:=i-1
      end;
    sp:=sp+1            { push stack }
  end       { if }
 end        { while }
end;  { QUICK SORT }

begin   { MAIN program }
 n:=max;
 randomize;
 for i:=1 to n do
   x[i]:= random(100);
 sort( x,n );
 write_arr
end.
```

## Программа 9. Численное интегрирование методом Ромберга (в.1).

```
program romb1;
{ integration by the romberg method }

const tol         = 1.0E-4;
var   done        : boolean;
      sum,upper,lower   : real;
function fx(x: real): real;
{ find f(x)= 1.0/x; watch out for x=0 }
begin
  fx:=1.0/x
end;
procedure romb(
            lower,upper,tol: real;
                  var ans: real);
{ numerical integration by the Romberg method }
{ function fx, name cannot be passed by Turbo Pascal}
var
      nx                : array[1..16] of integer;
      t                 : array[1..136] of real;
      done,error        : boolean;
      pieces,nt,i,ii,n,nn,
      l,ntra,k,m,j            : integer ;
      delta_x,c,sum,fotom,x   : real ;
begin
  done:=false;
  error:=false;
```

```
  pieces:=1;
  nx[1]:=1;
  delta_x:=(upper-lower)/pieces;
  c:=(fx(lower)+fx(upper))*0.5;
  t[1]:=delta_x*c;
  n:=1;
  nn:=2;
  sum:=c;
  repeat
    n:=n+1;
    fotom:=4.0;
    nx[n]:=nn;
    pieces:=pieces*2;
    l:=pieces-1;
    delta_x:=(upper-lower)/pieces;
    { compute trapezoidal sum for 2^(n-1)+1 points }
    for ii:=1 to (l+1) div 2 do
      begin
      i:=ii*2-1;
      x:=lower+i*delta_x;
      sum:=sum+fx(x)
      end;
    t[nn]:=delta_x*sum;
    write(pieces:5,t[nn]);
    ntra:=nx[n-1];
    k:=n-1;
    { compute n-th row of T array }
    for m:=1 to k do
      begin
      j:=nn+m;
      nt:=nx[n-1]+m-1;
      t[j]:=(fotom*t[j-1]-t[nt])/(fotom-1.0);
      fotom:=fotom*4.0
      end;
    writeln(j:4,t[j]);
    if n>4 then
      begin
      if t[nn+1]<>0.0 then
        if (abs(t[ntra+1]-t[nn+1])<=abs(t[nn+1]*tol))
          or (abs(t[nn-1]-t[j])<=abs(t[j]*tol)) then
            done:=true
        else if n>15 then
          begin
            done:=true;
            error:=true
          end
      end;  { if n>4 }
    nn:=j+1
  until done;
  ans:=t[j]
end;        { ROMBERG }

begin       { main program }
  ClrScr;
  lower:=1.0;
  upper:=9.0;
  writeln;
  romb(lower,upper,tol,sum);
  writeln;
  writeln(chr(7),'Area= ',sum)
end.
```

**Программа 10. Сортировка методом Шелла (в.2)**

```
program Shell_sort;
{ Shell-Metzner sort }
{ Adapted from 'Programming in pascal',
      P. Grogono, Addison-Wesley, 1980 }

const   max     = 80;

type    ary     = array[1..max]of real;

var     x         : ary;
        i,n       : integer;

procedure {shell} sort(var a: ary; n: integer);

var   done  : boolean;
      jump,i,j: integer;

procedure swap(var p,q: real);
var   hold  : real;

begin
  hold:=p;
  p:=q;
  q:=hold
end;        { swap }

begin
  jump:=n;
  while jump>1 do
    begin
      jump:=jump div 2;
      repeat
      done:=true;
      for j:=1 to n do
        begin
          i:=j+jump;
          if a[j]>a[i] then
            begin
            swap(a[j],a[i]);
            done:=false
          end           { if }
      end         { for }
      until done
  end             { while }
end;        { SORT }

procedure write_arr;
{ print out the answer }
var
  i: integer;

begin
  writeln;
  for i:=1 to n do
      write(x[i]:7:1,' ');
      writeln;
end;        { write_arr }

begin   { MAIN program }
 n:=max;
 randomize;
 for i:=1 to n do
   x[i]:= random(100);
 sort( x,n );
 write_arr
```

```
end.
```

**Программа 11. Вычисление функции ошибок распределения Гаусса (в.1).**

```
program erfd3;
{ evaluation of the gaussian error function }

var   x,er,ec           : real;
      done        : boolean;

function erf(x: real): real;
{ infinite series expansion of the Gaussian error function }

const sqrtpi            = 1.7724538;
      tol         = 1.0E-4;

var   x2,sum,sum1,term: real;
      i           : integer;

begin
      x2:=x*x;
      sum:=x;
      term:=x;
      i:=0;
      repeat
      i:=i+1;
      sum1:=sum;
      term:=2.0*term*x2/(1.0+2.0*i);
      sum:=term+sum1
      until term<tol*sum;
      erf:=2.0*sum*exp(-x2)/sqrtpi
end;  { erf }

function erfc(x: real): real;
{ complement of error function }
const sqrtpi            = 1.7724538;
      terms       = 12;

var   x2,u,v,sum  : real;
      i           : integer;
begin
  x2:=x*x;
  v:=1.0/(2.0*x2);
  u:=1.0+v*(terms+1.0);
  for i:=terms downto 1 do
    begin
      sum:=1.0+i*v/u;
      u:=sum
    end;
  erfc:=exp(-x2)/(x*sum*sqrtpi)
end;        { ercf }

begin       { main }
  ClrScr;
  done:=false;
  writeln;
  repeat
    write('Arg? ');
    readln(x);
    if x<0.0 then done:=true
    else
      begin
      if x=0.0 then
```

```
        begin
          er:=0.0;
          ec:=1.0
        end
      else
        begin
          if x<1.5 then
            begin
            er:=erf(x);
            ec:=1.0-er
            end
          else
            begin
            ec:=erfc(x);
            er:=1.0-ec
          end     { if }
      end;
      writeln('X= ',x:8:4,' Erf= ',er:12:8,', Erfc= ',ec:12)
      end   { if }
    until done
end.
```

*Замечание. Надо в исходном коде в части main добавить цикл, в котором значение аргумента x меняется от x=2 с шагом -1 до x=-1. Тогда будут проверены все части алгоритма.*

**Программа 12. Приближенная линеаризация опытных данных (в.1).**

Написать головную программу генерации двух массивов координат x,y точек плоскости с помощью датчиков случайных чисел и линеаризации полученных данных с помощью процедуры linfit.

```
procedure linfit2(x,y: ary;
             var y_calc: ary;
             var a,b:    real;
                  n:   integer);
{ fit a straight line (y_calc) through n sets of x and y pairs of points }

var   i           : integer;

      sum_x,sum_y,sum_xy,sum_x2,
      sum_y2,xi,yi,sxy,syy,
      sxx         : real;

begin             { linfit }
  sum_x:=0.0;
  sum_y:=0.0;
  sum_xy:=0.0;
  sum_x2:=0.0;
  sum_y2:=0.0;
  for i:=1 to n do
    begin
      xi:=x[i];
      yi:=y[i];
      sum_x:=sum_x+xi;
      sum_y:=sum_y+yi;
      sum_xy:=sum_xy+xi*yi;
      sum_x2:=sum_x2+xi*xi;
      sum_y2:=sum_y2+yi*yi;
    end;
  sxx:=sum_x2-sum_x*sum_x/n;
  sxy:=sum_xy-sum_x*sum_y/n;
  syy:=sum_y2-sum_y*sum_y/n;
```

```
  b:=sxy/sxx;
  a:=((sum_x2*sum_y-sum_x*sum_xy)/n)/sxx;
  correl_coef:=sxy/sqrt(sxx*syy);
  see:=sqrt((sum_y2-a*sum_y-b*sum_xy)/(n-2));
  sigma_b:=see/sqrt(sxx);
  sigma_a:=sigma_b*sqrt(sum_x2/n);

  for i:=1 to n do
    y_calc[i]:=a+b*x[i]
end;        { LINFIT }
```

**Программа 13. Решение системы уравнений методом Гаусса-Зейделя.**

```
program gausid;
{ pascal program to perform simultaneous solution }
{ by Gauss-Seidel }
{ procedure SEID is included }

const maxr  = 8;
      maxc  = 8;

type  ary   = array[1..maxr] of real;
      arys  = array[1..maxc] of real;
      ary2s = array[1..maxr,1..maxc] of real;

var   y     : ary;
      coef  : arys;
      a     : ary2s;
      n,m   : integer;
      first,
      error : boolean;


procedure write_data;
{ print out the answers }

var   i : integer;

begin
  for i:=1 to m do
    write(coef[i]:9:5);
  writeln
end;        { write_data }

procedure seid
      (a    : ary2s;
      y     : ary;
      var coef: arys;
      ncol  : integer;
       var error: boolean);
{ matrix solution by Gauss Seidel }

const tol   = 1.0E-4;
const max   = 100;
{ }

var   done  : boolean;
      i,j,k,l,n: integer;

      nextc,hold,sum,lambda,ab,big: real;
  { Relaxation factor lambda in range (0.0, 2.0)}

  lambda = 1.5;
```

```
n:=ncol;
error:=false;

for i:=1 to n-1 do
  begin
    big:=abs(a[i,i]);
    l:=i;
    for j:=i+1 to n do
    begin
    { search for largest element }
      ab:=abs(a[j,i]);
      if ab>big then
        begin
          big:=ab;
          l:=j
        end
    end;        { j-loop }
  if big=0.0 then error:=true
  else
    begin
    if l<>i then
      begin
      { interchange rows to put }
      { largest element on diagonal }
      for j:=1 to n do
        begin
          hold:=a[l,j];
          a[l,j]:=a[i,j];
          a[i,j]:=hold
        end;
      hold:=y[l];
      y[l]:=y[i];
      y[i]:=hold
    end   { if l<>i }
    end   { if big }
  end;    { i-loop }
if a[n,n]=0.0 then error:=true
else
  begin
    for i:=1 to n do
    coef[i]:=0.0;           { initial guess }
    i:=0;
    repeat
    i:=i+1;
    done:=true;
    for j:=1 to n do
      begin
        sum:=y[j];
        for k:=1 to n do
          if j<>k then
          sum:=sum-a[j,k]*coef[k];
        nextc:=sum/a[j,j];
        if abs(nextc-coef[j])>tol then
          begin
          done:=false;
          if nextc*coef[j]<0.0 then
            nextc:=(coef[j]+nextc)*0.5
          end;
        coef[j]:=lambda*nextc+(1.0-lambda)*coef[j];
        writeln(i:4,',coef(',j,')=',coef[j])
    end         { j-loop }
    until done or (i>max)
end;      { if a[n,n]=0 }
if i>max then error:=true;
if error then writeln('ERROR: Matrix is singular')
```

```
end;        { SEID }

begin       { MAIN program }
  first:=true;
  writeln;
  writeln('Simultaneous solution by Gauss-Seidel');

{Исходные данные задаются как постоянные}

n = 4; m = 4;

a[1,1] = 20.9; a[1,2] = 1.2; a[1,3] = 2.1; a[1,4] = 0.9;
a[2,1] = 1.2; a[2,2] = 21.2; a[2,3] = 1.5; a[2,4] = 2.5;
a[3,1] = 2.1; a[3,2] = 1.5; a[3,3] = 19.8; a[3,4] = 1.3;
a[4,1] = 0.9; a[4,2] = 2.5; a[4,3] = 1.3; a[4,4] = 32.1;

y[1] = 21.7;   y[2] = 27.5;    y[3] = 28.8 ;   y[4] = 49.7


      if n>1 then
      begin
      seid(a,y,coef,n,error);
      if not error then write_data
      end
end.
```

## Программа 14. Вычисление функции Бесселя 2-го рода (в.2)

```
program bessy2;
{ evaluation of Bessel function of the second kind }
uses crt;

var   x,ordr      : real;
      done  : boolean;

function bessy(x,n: real): real;
{ cylindical bessel function of the second kind }
const small = 1.0E-8;
      euler = 0.57721566;
      pi    = 3.1415926;
      pi2   = 0.63661977;     { 2/pi }
var   j     : integer;

      x2,sum,sum2,t,t2,
      ts,term,xx,y0,y1,
      ya,yb,yc,ans,a,b,
      sina,cosa         : real;

begin             { function bessy }
  if x<12 then
    begin
      xx:=0.5*x;
      x2:=xx*xx;
      t:=ln(xx)+euler;
      sum:=0.0;
      term:=t;
      y0:=t;
      j:=0;
      repeat
      j:=j+1;
      if j<>1 then sum:=sum+1/(j-1);
      ts:=t-sum;
      term:=-x2*term/(j*j)*(1-1/(j*ts));
      y0:=y0+term
```

```
      until abs(term)<small;
      term:=xx*(t-0.5);
      sum:=0.0;
      y1:=term;
      j:=1;
      repeat
      j:=j+1;
      sum:=sum+1/(j-1);
      ts:=t-sum;
      term:=(-x2*term)/(j*(j-1))*((ts-0.5/j)/(ts+0.5/(j-1)));
      y1:=y1+term
      until abs(term)<small;
      y0:=pi2*y0;
      y1:=pi2*(y1-1/x);
      if n=0.0 then ans:=y0
      else if n=1.0 then ans:=y1
      else
      begin       { find y by recursion }
        ts:=2.0/x;
        ya:=y0;
        yb:=y1;
        for j:=2 to trunc(n+0.01) do
          begin
            yc:=ts*(j-1)*yb-ya;
            ya:=yb;
            yb:=yc
          end;
        ans:=yc
      end;
      bessy:=ans;
    end           { x<12 }
  else            { x>11, asymtotic expansion }
    bessy:=sqrt(2/(pi*x))*sin(x-pi/4-n*pi/2)
end;  { function bessy }

begin
  ClrScr;
  done:=false;
  writeln;
  repeat
    write('Order? ');
    readln(ordr);
    if ordr<0.0 then done:=true
    else
      begin
      repeat
        write('Arg? ');
        readln(x)
      until x>=0.0;
      writeln('Y Bessel is ',bessy(x,ordr))
    end           { if }
  until done
end.
```

Исх.данные: x = 5 ,ordr = 10

**Программа 15. Интегрирование методом трапеций (в.2).**

```
program trap2;
{ integration by the trapezoidal rule }

const tol         = 1.0E-6;
var   sum,upper,lower   : real;

function fx(x: real): real;
```

```
{ find f(x)=1/sqrt(x) }
{ watch out for x=0 ! }
begin
  fx:=1.0/ sqrt(x)
end;

procedure trapez(lower,upper,tol: real;
            var sum           : real);
{ numerical integration by the trapezoid method }
{ function is FX, limits are LOWER and UPPER }
{ with number of regions equal to PIECES }
{ fixed partition is DELTA_X, answer is SUM }

var   pieces,i                : integer;
      x,delta_x,end_sum,mid_sum,sum1      : real;
begin
  pieces:=1;
  delta_x:=(upper-lower)/pieces;
  end_sum:=fx(lower)+fx(upper);
  sum:=end_sum*delta_x/2.0;
  writeln('    1',sum);
  mid_sum:=0.0;
  repeat
    pieces:=pieces*2;
    sum1:=sum;
    delta_x:=(upper-lower)/pieces;
    for i:=1 to pieces div 2 do
    begin
      x:=lower+delta_x*(2.0*i-1.0);
      mid_sum:=mid_sum+fx(x)
    end;
  sum:=(end_sum+2.0*mid_sum)*delta_x*0.5;
  writeln(pieces:5,sum)
  until abs(sum-sum1)<=abs(tol*sum)
end;        { TRAPEZ }

begin       { main program }
  ClrScr;
  lower:=1.0;
  upper:=9.0;
  writeln;
  trapez(lower,upper,tol,sum);
  writeln;
  writeln(chr(7),'area=',sum)
end.
```

**Программа 16. Вычисление функции ошибок распределения Гаусса (в.2)**.

```
program erfd4;

{ evaluation of the gaussian error function }

var   x,er,ec           : real;
      done        : boolean;

function erf(x: real): real;
{ infinite series expansion of the Gaussian error function }

const sqrtpi            = 1.7724538;
      t2          = 0.66666667;
      t3          = 0.66666667;
      t4          = 0.07619048;
      t5          = 0.01693122;
      t6          = 3.078403E-3;
```

```
      t7          = 4.736005E-4;
      t8          = 6.314673E-5;
      t9          = 7.429027E-6;
      t10         = 7.820028E-7;
      t11         = 7.447646E-8;
      t12         = 6.476214E-9;

var   x2,sum            : real;
      i           : integer;

begin
  x2:=x*x;
  sum:=t5+x2*(t6+x2*(t7+x2*(t8+x2*(t9+x2*(t10+x2*(t11+x2*t12))))));
  erf:=2.0*exp(-x2)/sqrtpi*(x*(1+x2*(t2+x2*(t3+x2*(t4+x2*sum)))))
end;  { function erf }

function erfc(x: real): real;
{ complement of error function }
const sqrtpi            = 1.7724538;

var   x2,v,sum    : real;

begin
  x2:=x*x;
  v:=1.0/(2.0*x2);
  sum:=v/(1+8*v/(1+9*v/(1+10*v/(1+11*v/(1+12*v)))));
  sum:=v/(1+3*v/(1+4*v/(1+5*v/(1+6*v/(1+7*sum)))));
  erfc:=1.0/(exp(x2)*x*sqrtpi*(1+v/(1+2*sum)))
end;        { function ercf }

begin       { main }
  ClrScr;
  done:=false;
  writeln;
  repeat
    write('Arg? ');
    readln(x);
    if x<0.0 then done:=true
    else
      begin
      if x=0.0 then
        begin
          er:=0.0;
          ec:=1.0
        end
      else
        begin
          if x<1.5 then
            begin
            er:=erf(x);
            ec:=1.0-er
            end
          else
            begin
            ec:=erfc(x);
            er:=1.0-ec
          end     { if }
      end;
      writeln('X= ',x:8:4,', Erf= ',er:12,', Erfc= ',ec:12)
      end   { if }
    until done
end.
```

*Замечание. Надо в исходном коде в части main добавить цикл, в котором значение аргумента x меняется от x=2 с шагом -1 до x=-1. Тогда будут проверены*

*все части алгоритма.*

**Программа 17. Добавить головную программу и сравнить эффективность двух процедур сортировки.**

```
program bubble_sort1-2;
{ bubble sorting - vers.1 without procedure, vers.2 with swap-procedure }

const   max     = 5000;

type    ary     = array[1..max]of real;

var     x         : ary;
        i,n       : integer;

procedure { bubble- } sort1(var a: ary; n: integer);
var   i,j   : integer;
      hold  : real;

begin       { procedure sort1 }
  for i:=1 to n-1 do
    for j:=i+1 to n do
      begin
      if a[i]>a[j] then
        begin
          hold:=a[i];
          a[i]:=a[j];
          a[j]:=hold
        end
    end           { for }
end;        { procedure sort1 }



procedure {bubble} sort2(var a: ary; n: integer);

var   no_change   : boolean;
      j           : integer;

procedure swap(p,q: real);
var   hold  : real;
begin
  hold:=p;
  p:=q;
  q:=hold
end;        { swap }

begin       { procedure sort2 }
  repeat
    no_change:=true;
    for j:=1 to n-1 do
      begin
      if a[j]>a[j+1] then
        begin
          swap(a[j],a[j+1]);
          no_change:=false
        end
      end   { for }
  until no_change
end;  { procedure sort2 }
```

**Программа 18. Численное интегрирование методом Ромберга (в.2).**

```
program romb3;
{ integration by the romberg method }

const tol         = 1.0E-4;
var   done        : boolean;
      sumt        : real;
      sum,upper,lower   : real;

function fx(x: real): real;
{ find f(x)= 1/sqrt(x); watch out for x=0 }
begin
  fx:=1.0/sqrt(x)
end;

procedure romb(
            lower,upper,tol: real;
                  var ans: real);
{ numerical integration by the Romberg method }
var
      nx                : array[1..16] of integer;
      t                 : array[1..136] of real;
      done,error        : boolean;
      pieces,nt,i,ii,n,nn,
      l,ntra,k,m,j            : integer ;
      delta_x,c,sum,fotom,x   : real ;
begin
  done:=false;
  error:=false;
  pieces:=1;
  nx[1]:=1;
  delta_x:=(upper-lower)/pieces;
  c:=(fx(lower)+fx(upper))*0.5;
  t[1]:=delta_x*c;
  n:=1;
  nn:=2;
  sum:=c;
  repeat
    n:=n+1;
    fotom:=4.0;
    nx[n]:=nn;
    pieces:=pieces*2;
    l:=pieces-1;
    delta_x:=(upper-lower)/pieces;
    { compute trapezoidal sum for 2^(n-1)+1 points }
    for ii:=1 to (l+1) div 2 do
      begin
      i:=ii*2-1;
      x:=lower+i*delta_x;
      sum:=sum+fx(x)
      end;
    t[nn]:=delta_x*sum;
    ntra:=nx[n-1];
    k:=n-1;
    { compute n-th row of T array }
    for m:=1 to k do
      begin
      j:=nn+m;
      nt:=nx[n-1]+m-1;
      t[j]:=(fotom*t[j-1]-t[nt])/(fotom-1.0);
      fotom:=fotom*4.0
```

```
      end;
    if n>4 then
      begin
      if t[nn+1]<>0.0 then
        if (abs(t[ntra+1]-t[nn+1])<=abs(t[nn+1]*tol))
          or (abs(t[nn-1]-t[j])<=abs(t[j]*tol)) then
            done:=true
        else if n>15 then
          begin
            done:=true;
            error:=true
          end
      end;  { if n>4 }
    nn:=j+1
  until done;
  ans:=t[j]
end;        { ROMBERG }

begin       { main program }
  ClrScr;
  lower:=0.1;
  upper:=1.0;
  writeln;
  sumt:=0.0;
  writeln(' new area        total area           lower          upper limits');
  repeat
    romb(lower,upper,tol,sum);
    upper:=lower;
    lower:=0.1*upper;
    sumt:=sumt+sum;
    writeln(sum:9:6,'    ',sumt:9:5,'    ',lower,'    ',upper)
  until abs(sum)<tol
end.
```

**Программа 19. Приближенная линеаризация опытных данных (в.2).**

Написать головную программу генерации двух массивов координат x,y точек плоскости с помощью датчиков случайных чисел и линеаризации полученных данных с помощью процедуры linfit.

```
procedure linfit1(x,y: ary;
             var y_calc: ary;
             var a,b:    real;
                  n:   integer);
{ fit a straight line (y_calc) through n sets of x and y pairs of points }

var   i           : integer;

      sum_x,sum_y,sum_xy,sum_x2,
      sum_y2,xi,yi,sxy,sxx,
      syy         : real;

begin             { linfit }
  sum_x:=0.0;
  sum_y:=0.0;
  sum_xy:=0.0;
  sum_x2:=0.0;
  sum_y2:=0.0;
  for i:=1 to n do
    begin
      xi:=x[i];
      yi:=y[i];
      sum_x:=sum_x+xi;
```

```
      sum_y:=sum_y+yi;
      sum_xy:=sum_xy+xi*yi;
      sum_x2:=sum_x2+xi*xi;
      sum_y2:=sum_y2+yi*yi;
    end;
  sxx:=sum_x2-sum_x*sum_x/n;
  sxy:=sum_xy-sum_x*sum_y/n;
  syy:=sum_y2-sum_y*sum_y/n;
  b:=sxy/sxx;
  a:=((sum_x2*sum_y-sum_x*sum_xy)/n)/sxx;
  for i:=1 to n do
    y_calc[i]:=a+b*x[i]
end;        { LINFIT }
```

**Программа 20. Быстрая сортировка (нерекурсивный вариант) – в.2.**

```
const   max     = 200;

type    ary     = array[1..max]of real;

var     x         : ary;
        i,n       : integer;

procedure sort(var x: ary; n: integer);
{ a NONRECURSIVE quicksort routine }
{ Adapted from 'Software-Tools',
      B.Kernighan, Addison Wesley, 1976 }

var   left,right  : array[1..50] of integer;
      i,j,sp,mid  : integer;
      pivot       : real;

procedure swap(var p,q: real);
var   hold  : real;

begin
  hold:=p;
  p:=q;
  q:=hold
end;        { swap }


begin
  left[1]:=1;
  right[1]:=n;
  sp:=1;
  while sp>0 do
    begin
      if left[sp]>=right[sp] then sp:=sp-1
      else
      begin
        i:=left[sp];
        j:=right[sp];
        pivot:=x[j];
        mid:=(i+j)div 2;
        if (j-i)>5 then
          if ((x[mid]<pivot)and(x[mid]>x[i]))
            or
            ((x[mid]>pivot)and(x[mid]<x[i]))
              then swap(x[mid],x[j])
          else
            if((x[i]<x[mid])and(x[i]>pivot))
            or ((x[i]>x[mid])and(x[i]<pivot))
              then swap(x[i],x[j]);
```

```
    pivot:=x[j];
    while i<j do
      begin
      while x[i]<pivot do
        i:=i+1;
      j:=j-1;
      while (i<j)and(pivot<x[j]) do
        j:=j-1;
      if i<j then swap(x[i],x[j])
    end;    { while }
    j:=right[sp]; { pivot to i }
    swap(x[i],x[j]);
    if i-left[sp]>=right[sp]-i then
      begin       { put shorter part first }
      left[sp+1]:=left[sp];
      right[sp+1]:=i-1;
      left[sp]:=i+1
      end
    else
      begin
      left[sp+1]:=i+1;
      right[sp+1]:=right[sp];
      right[sp]:=i-1
      end;
    sp:=sp+1            { push stack }
  end       { if }
 end        { while }
end;  { QUICK SORT }

begin   { MAIN program }
 n:=max;
 randomize;
 for i:=1 to n do
   x[i]:= random(100);
 sort( x,n );
 write_arr
end.
```

**Программа 21. Умножение матриц.**

```
program matr1;
{ pascal program to perform matrix multiplication }

const rmax  = 9;
      cmax  = 3;

type  ary   = array[1..rmax] of real;
      arys  = array[1..cmax] of real;
      ary2  = array[1..rmax,1..cmax] of real;
      ary2s = array[1..cmax,1..cmax] of real;
var   y           : ary;
      g           : arys;
      x           : ary2;
      a           : ary2s;
      nrow,ncol   : integer;

procedure get_data(var x: ary2;
               var y: ary;
         var nrow,ncol: integer);
{ get the values for nrow, ncol, and arrays x,y }
      var   i,j   : integer;

begin
  nrow:=5;  {эти значения не следует считать }
```

```
  ncol:=3;  { фиксированными - они могут изменяться}
  for i:=1 to nrow do
    begin
      x[i,1]:=1;
      for j:=2 to ncol do
        x[i,j]:=i*x[i,j-1];
      y[i]:=2*i
    end
end;        { procedure get_data }
procedure write_data;

{ print out the answeres }
var
  i,j : integer;
begin
  ClrScr;
  writeln;
  writeln('          X             Y');
  for i:=1 to nrow do
    begin
      for j:=1 to ncol do
        write(x[i,j]:7:1,' ');
      writeln(':',y[i]:7:1)
    end;
  writeln('          A             G');
  for i:=1 to ncol do
    begin
      for j:=1 to ncol do
        write(a[i,j]:7:1,' ');
      writeln(':',g[i]:7:1)
    end
end;        { write_data }

procedure square(x: ary2;
             y: ary;
           var a: ary2s;
           var g: arys;
       nrow,ncol: integer);

{ matrix multiplication routine }
{ a= transpose x times x }
{ g= y times x }

var
 i,k,l      : integer;

begin       { square }
  for k:=1 to ncol do
    begin
      for l:=1 to k do
        begin
          a[k,l]:=0;
          for i:=1 to nrow do
          begin
              a[k,l]:=a[k,l]+x[i,l]*x[i,k];
              if k<>l then a[l,k]:=a[k,l]
          end
       end;       { l-loop }
       g[k]:=0;
       for i:=1 to nrow do
       g[k]:=g[k]+y[i]*x[i,k]
  end { k-loop }
end;  { square }

begin { MAIN program }
```

```
  get_data(x,y,nrow,ncol);
  square(x,y,a,g,nrow,ncol);
  write_data
end.
```